

Project Title: Deepfake Detection Using Transfer Learning Approaches

Names

Liam Merz Hoffmeister, James Serrano, Christina Sun

Abstract

This report presents our implementation and evaluation of various deepfake detection models. We used a simple convolutional neural network (CNN) as a baseline to distinguish between genuine and AI-generated fake videos, with a focus on the Celeb-DF (v2) dataset. We then expanded our approach and compared it against several other CNN-based architectures via transfer learning, and we also created a new model that used feature extraction on specific criteria deep fakes are known to struggle with. Our best model achieved 79.0% accuracy and 0.764 AUC, significantly overperforming our initial expectations. This work contributes to the broader effort of maintaining media integrity in an era of increasingly sophisticated AI-generated content.

1. Introduction

The rapid advancement of artificial intelligence (AI) has led to a proliferation of deepfake technology, which can generate highly realistic but fabricated media. Deepfakes, especially videos, have increasingly been used for misinformation campaigns, including the impersonation of political figures and public personalities. These manipulated images and videos pose significant threats to public trust and democratic institutions. Our project aimed to develop effective detection methods for these deepfakes using various machine learning approaches. We implemented two distinct methodologies: one based on deep learning and transfer learning, and another based on traditional machine learning with feature engineering.

In the first approach, we employed a convolutional neural network (CNN) as a baseline. We then chose to incorporate transfer learning to utilize pre-trained models. This approach allows us to take advantage of large, pre-trained networks that have been optimized on massive datasets, reducing the need for extensive data and computational resources. By using these models as a starting point, we can fine-tune them for our specific task while avoiding the need for training a network from scratch, which can be both time-consuming and resource-intensive. The CNN-based transfer learning model focused on identifying patterns in pixel data that distinguish real videos from manipulated ones, taking advantage of deep learning's ability to automatically learn complex features in large datasets.

In the second approach, we explored traditional machine learning techniques, such as logistic regression, using handcrafted features derived from known deepfake artifacts. Our DeepfakeDetector class leverages temporal consistency analysis, texture pattern detection, physiological signal monitoring, and other artifacts to identify manipulated facial content. We focus on distinguishing real and AI-generated videos with a particular emphasis on facial manipulations, as these are often the key regions of tampering. This method complements the CNN-based model by providing a more targeted approach to detecting manipulation.

The primary challenges we faced included managing computational limitations, mitigating dataset imbalance, effectively preprocessing facial regions, and handling the subtle artifacts present in high-quality deepfake videos.

2. Literature Review

Deepfakes, a portmanteau of "deep learning" and "fake," refer to synthetic media in which a person's likeness is manipulated using artificial intelligence techniques. As deepfake generation tools become increasingly accessible and sophisticated, reliable detection systems are needed to mitigate potential threats in media authenticity, political misinformation, and cybersecurity. Recent advances in computer vision and deep learning have made transfer learning-based approaches especially effective in tackling deepfake detection tasks. Transfer learning allows researchers to leverage pre-trained models on large-scale datasets such as ImageNet (Russakovsky et al., 2015), reducing training time and computational requirements while improving generalization on limited deepfake datasets (Lin et al., 2023).

2.1 Vision-Based Deepfake Detection Methods

Recent advances in vision-based models, particularly Vision Transformers (ViTs), have shown strong performance in deepfake detection. Unlike traditional Convolutional Neural Networks (CNNs), ViTs use self-attention to capture long-range dependencies in images, enabling them to identify subtle, global facial inconsistencies often missed by local convolutional filters (Carion et al., 2020; Dosovitskiy et al., 2021). Studies have shown that ViTs fine-tuned on facial regions outperform CNN-based approaches on high-resolution datasets (Dosovitskiy et al., 2015), although their computational demands and need for large datasets can be limiting in low-resource settings.

Other models focus on detecting visual inconsistencies introduced by manipulation. These fall into three main categories: temporal (e.g., unnatural motion between frames), spatial (e.g., misalignment or blending artifacts), and physiological (e.g., irregular blinking). These cues are useful across both handcrafted and deep learning-based approaches, especially in video analysis (Rössler et al., 2019).

Hybrid models that combine CNNs for spatial feature extraction with Recurrent Neural Networks (RNNs), such as LSTMs or GRUs, for temporal modeling have also proven effective (D'Avino et

al., 2017). These models can track frame-level inconsistencies over time but often introduce additional complexity and latency, limiting their suitability for real-time or mobile applications.

To address efficiency concerns, lightweight CNN architectures like EfficientNet have emerged as strong candidates (Tan & Le, 2020). EfficientNet scales depth, width, and resolution in a balanced manner, and when paired with attention mechanisms, it can focus on facial regions most prone to manipulation while keeping computational costs low. This makes it especially suitable for deployment on resource-constrained platforms, including edge devices and tools like Google Colab.

2.2 Transfer Learning and Model Selection

Given the limited size and variability of deepfake datasets, transfer learning is a practical strategy that enables the reuse of pre-trained image classification models (Yosinski et al., 2014). In this study, we evaluated three CNN-based architectures—EfficientNet-B0, MobileNet_v2, and ResNet18—based on their trade-offs between performance and efficiency.

EfficientNet-B0, the smallest in its family, offers a strong balance between accuracy and computational cost. Pre-trained on ImageNet, it has shown competitive results with fewer parameters and FLOPs, and is easily accessible through open-source libraries like lukemelas/EfficientNet-PyTorch (Tan & Le, 2020).

MobileNet_v2 is optimized for speed and low memory usage, making it ideal for mobile and embedded applications. Its use of depthwise separable convolutions and inverted residual blocks allows for fast inference without significantly sacrificing accuracy (Sandler et al., 2018).

ResNet18, a well-established baseline with residual connections to ease training, remains a popular choice for benchmarking newer architectures due to its simplicity and reliability (He et al., 2016).

All three models were fine-tuned on deepfake datasets using a transfer learning approach. Training began with frozen pre-trained weights, followed by gradual unfreezing of deeper layers to adapt the models to the domain while minimizing overfitting.

2.3 Feature Engineering

Deepfake detection is a rapidly evolving area, and although progress has been made, there are still several challenges that detectors face when trying to identify synthetic content.

Deepfakes, especially those generated using Generative Adversarial Networks (GANs), exhibit characteristic artifacts such as overly smooth textures, unnatural skin tones, or inconsistent lighting and shadows. While low-quality deepfakes show prominent facial distortions or artifacts (e.g., pixelation), high-quality deepfakes generated by recent models like StyleGAN2 or ProGAN have learned to reduce these imperfections, making it harder for detectors to spot subtle inconsistencies, especially on skin and hair (Afchar et al., 2018; Rössler et al., 2019).

Detectors also struggle with temporal inconsistencies, particularly with dynamic facial features like eye movements, mouth shape during speech, and blinking behavior. GANs struggle to maintain temporal coherence, resulting in unnatural transitions between frames, particularly for regions like the mouth, eyes, and jawline (Zhou et al., 2017).

Another hallmark of deepfakes is unnatural blinking patterns or inconsistent eye movements. GANs have historically struggled with generating natural eye blinking and gaze transitions, making them a potential giveaway for synthetic faces. While these models have improved, subtle inconsistencies, particularly in frame-dependent eye movements, can still be challenging for detectors (Sharma & Dwivedi, 2025).

Inconsistent lighting or shadows is a common artifact in low-quality deepfakes, especially in older GAN models. However, state-of-the-art deepfake models have learned to better match real-world lighting and shadow patterns, making this feature less of a giveaway in high-quality fakes. Deepfake detectors often struggle to detect lighting mismatches in real-time, as subtle lighting changes (especially around the eyes, mouth, and face outline) can be difficult to quantify across varying environments (Wu et al., 2022).

Lastly, an interesting and emerging area of deepfake detection involves tracking physiological signals, such as pulse (via remote photoplethysmography, rPPG) and other subtle skin tone variations that reflect heart rate. These signals are often absent in synthetic faces, which cannot replicate the subtle physiological changes of a person's body. Despite the promising nature of this feature, current deepfake detectors still struggle with accurately extracting physiological signals due to noise, lighting conditions, or motion artifacts (Hernandez-Ortega et al., 2020).

3. Dataset and Downsampling

We initially utilized the Celeb-DF (v2) dataset (*Celeb DF (V2)*, n.d.), which contains 5,639 high-quality deepfake videos of celebrities alongside genuine video samples. This dataset was chosen because it contains high-quality deepfakes that are challenging to detect. Public figures, like celebrities and politicians, are frequently targeted for deepfake videos due to their visibility and influence. Furthermore, this dataset has videos that have a similar visual quality to those that are circulated online. In contrast to older datasets such as FaceForensics++ or the original Celeb-DF (v1), the v2 dataset exhibits fewer easily exploitable visual glitches like flickering, mouth warping, or ghosting artifacts, which makes detection more difficult and therefore more realistic (Li et al., 2020).

Because processing every frame of each video would be computationally prohibitive, we chose to downsample the data for every model. We randomly sampled one frame every 3 seconds for every video. This approach trades temporal depth for computational tractability, but it still enables meaningful classification, particularly if the frame is representative or includes subtle visual anomalies introduced by deepfake generation (e.g., inconsistent eye geometry or compression artifacts). Frames were extracted using OpenCV and stored under consistent naming conventions, which were then used to construct a label map via a CSV file containing

filename and label columns. The label was binary: 0 for real (unaltered) videos, and 1 for fake (deepfake) videos. We then realized that the initial Celeb-DF (v2) dataset contains a training imbalance of 86% fake videos and 14% real videos. This ended up creating a model trained to label every frame as fake and still achieve high accuracy. To remedy this, we randomly omitted 2/3rds of the fake videos to achieve a more even balance of 67% fake videos to 33% real videos.

4. Methodology and Implementation Details

To address the problem of deepfake detection, we implemented and compared three distinct approaches: (0) simple CNN as a baseline, (1) deep learning via transfer learning using various pretrained models, and (2) initial feature extraction, (3) more complex feature extraction. Each method involves a structured pipeline from data preprocessing to model training, evaluation, and comparison.

4.1 Deep Learning with Transfer Learning

We implemented a deep learning-based approach using transfer learning, adapting CNN backbones for binary classification on our frame dataset. In this approach, we used the entire image for training and did not explicitly extract features first. We experimented with three pre-trained CNN architectures:

1. **EfficientNet-B0**: A compact but efficient model with balanced depth, width, and resolution
2. **MobileNet-V2**: A lightweight model designed for mobile applications
3. **ResNet18**: A residual network with skip connections to address vanishing gradient problems

We also implemented a SimpleCNN as a baseline model to compare against the transfer learning approaches. This decision was made to provide a reference point for evaluating the more complex models.

All models were loaded using PyTorch's torchvision.models, initialized with pretrained ImageNet weights. For each architecture, we replaced the classification head with custom layers for binary classification, initially froze the backbone weights to train only the classification head, and then implemented a fine-tuning phase where we unfroze the last few backbone layers.

Frames were resized and normalized before being passed to the models. Data augmentation techniques including random horizontal flips, slight rotations, color jittering, and affine transformations were applied to increase robustness. We used face extraction from video frames using established face detection libraries.

Our training approach included:

- Transfer learning from pre-trained models

- Binary cross-entropy loss with logits as the objective function
- AdamW optimizer with weight decay to prevent overfitting
- Learning rate scheduling with ReduceLROnPlateau
- Early stopping to prevent overfitting
- Dropout layers in the classification head

The hyperparameter configurations used for our models were stored in `MLProjConfigs.yaml` for reproducibility and ease of experimentation. Across all experiments, we used a batch size of 32 and trained the model for 5 epochs. The 5 epochs were decided based on a limitation due to the amount of time it took to train the model, which was a major hurdle. The total number of epochs trained was severely limited by the training time required. The initial learning rate was 0.001 and dynamically adjusted via the `ReduceLROnPlateau` scheduler, which reduced the learning rate by a factor of 0.5 if the validation loss did not improve for 3 consecutive epochs. For the transfer learning models, additional configurations included a dropout rate of 0.5 in the classification head, AdamW optimizer with a weight decay of $1e-4$. These settings were implemented using PyTorch.

4.2 Feature-Based Classification Pipeline

In our classical pipeline we manually extracted a fixed-length feature vector from each image and used traditional machine learning classifiers to detect whether the image was from a real or fake video. The selection of features were informed by literature on image quality assessment, deepfake detection artifacts, and human physiology. The features we implemented include:

1. **Grayscale Mean & Variance:** These global statistics serve as proxies for lighting uniformity and image contrast. Manipulated images may contain unnatural lighting distributions.
2. **RGB Histograms:** Each color channel (R, G, B) was binned into 32 values, normalized, and concatenated. These capture color intensity distributions which may be subtly altered in GAN-generated faces.
3. **Edge Density:** Using the Canny edge detector, we computed the proportion of edge pixels in the image. Over-smoothing or abnormal sharpness—often indicative of manipulations—can affect this measure.
4. **Laplacian Variance:** This captures high-frequency content and serves as an indirect proxy for JPEG compression artifacts. Poor compression fidelity or synthetic blending can result in abnormally low Laplacian variance.
5. **Eye Aspect Ratio (EAR):** Using dlib's 68-point facial landmark model, we computed the EAR—a geometric descriptor of eye openness. Since GANs often fail to realistically model blinking or eye structure, this value can help discriminate against fake images.

Each image yielded a total of 101 features: 2 grayscale + 96 RGB histogram bins + 1 edge density + 1 Laplacian variance + 1 EAR. All feature extraction was implemented in the function `extract_frame_features()` in `newmodels.py`. This function uses `cv2`, `numpy`, and `dlib` to load and process each image. Failed extractions due to unreadable files or undetected landmarks were logged and skipped. The resulting dataset consisted of a filtered list of valid (features, label)

pairs. We then created NumPy arrays X and y , storing features and labels, respectively. These were used for classifier training. Two classifiers were trained and compared:

1. Logistic Regression with L2 regularization and maximum iteration limit set to 1000
2. Support Vector Classifier (SVC) with RBF kernel and probability=True for AUC computation

A stratified 80/20 train-test split was used to ensure class balance.

The cost function used for logistic regression is shown below, where $h_\theta(x) = \sigma(\theta^T x)$, σ is the sigmoid function, and λ is the regularization parameter. The logistic regression model was optimized using the L-BFGS algorithm (a quasi-Newton method), which is the default solver in `sklearn.linear_model.LogisticRegression`. The objective function minimized for logistic regression was binary cross-entropy loss.

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(h_\theta(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)})) \right] + \lambda \|\theta\|^2$$

The objective function used for SVM and its constraints are shown below, where $\phi(x)$ maps inputs into a high-dimensional feature space using the RBF kernel, C is the regularization parameter, and ξ_i are slack variables for soft-margin violation. The support vector classifier was trained using `sklearn.svm.SVC`, which employs the Sequential Minimal Optimization (SMO) algorithm under the hood to solve the quadratic programming problem associated with soft-margin SVM. For the SVM, the hinge loss was minimized with L2 regularization. The RBF kernel was used to allow nonlinear separation, with the regularization parameter C set to 1.0.

$$\begin{aligned} \min_{w, b, \zeta} \quad & \frac{1}{2} w^T w + C \sum_{i=1}^n \zeta_i \\ \text{subject to} \quad & y_i (w^T \phi(x_i) + b) \geq 1 - \zeta_i, \\ & \zeta_i \geq 0, i = 1, \dots, n \end{aligned}$$

4.3 Extended Feature-Based Logistic Regression Classification

While the approach in Section 4.2 extracts features from individual frames, this extended approach incorporates temporal information across video sequences to detect motion and physiological inconsistencies that are only visible when analyzing multiple frames. This temporal analysis targets artifacts that emerge from frame-to-frame inconsistencies in deepfake generation.

Our DeepfakeDetector class extracts three categories of temporal features:

- **Facial Landmarks:** For each frame, 68 facial landmarks were detected and stored as coordinate arrays. These landmarks form the basis for calculating facial motion, geometry, and expression patterns over time.
- **Landmark Velocities:** Frame-to-frame landmark displacements were computed to derive both average and maximum landmark velocities. These capture inconsistencies in facial motion—especially in regions like the mouth and eyes—where GANs often struggle to maintain temporal coherence.
- **Eye Aspect Ratios (EAR):** Continuously monitored across frames to track blinking behavior. This temporal signal was used to identify blink frequency and duration, aiding in the detection of unrealistic blinking patterns common in synthesized videos.
- **Head Poses:** Using 2D landmarks, we estimated 3D head orientation in terms of pitch, yaw, and roll. Sudden or anatomically improbable head movements may indicate manipulation or poor tracking in synthetic content.
- **Blink Events:** Detected using thresholds on the EAR signal. Each blink event was logged by its frame index, enabling statistical analysis of blink rate and blink interval variability.
- **Texture Descriptors:** Local Binary Patterns (LBP) were applied to facial regions to describe fine-grained texture. Unnatural or overly smooth textures—typical artifacts of generative models—are reflected in LBP histogram irregularities.
- **Noise Patterns:** Noise statistics, such as local variance and spectral content, were computed from face patches to detect inconsistencies introduced by image synthesis or blending.
- **Pulse Signals:** Color variations in skin tone were tracked frame-by-frame to estimate pulse-like signals (rPPG). Irregular or missing pulse rhythms suggest synthetic origins, as GANs often fail to replicate subtle physiological signals.
- **Face Bounding Boxes:** For each frame, the coordinates of the detected face region were stored to spatially anchor all feature extraction steps.

After extraction, these features were compiled into structured representations per video. Temporal features such as EAR, landmark velocities, and pulse signals were interpolated to a fixed number of frames per video to enable comparison across samples. Feature aggregation (e.g., mean, variance, frequency-domain statistics) was then applied to produce a fixed-length feature vector per video. These vectors were used to train a separate Logistic regression classifier, which we compare against the static-feature and CNN-based pipelines.

The combination of motion dynamics, texture irregularities, and physiological cues aims to capture the temporal and biological inconsistencies that remain a challenge for most generative models. This method complements spatial pipelines by leveraging information across time and modality.

4.3.1 Mathematical Formulations and Optimization

For these facial feature extraction methods, we employed several mathematical formulations:

Eye Aspect Ratio (EAR) was calculated using the formula:

$$EAR = \frac{||p_2 - p_6|| + ||p_3 - p_5||}{2 \cdot ||p_1 - p_4||}$$

Where $p_1, p_2, \dots, p_6$ are the 2D landmark coordinates of the eye.

Landmark Velocities were computed as the Euclidean distance between corresponding landmarks in consecutive frames:

$$v_i(t) = ||p_i(t) - p_i(t - 1)||_2$$

Where $p_i(t)$ is the position of landmark i at frame t . The average and maximum velocities were then calculated as:

$$v_{avg}(t) = \frac{1}{n} \sum_{i=1}^n v_i(t) \quad v_{max}(t) = \max_{i \in 1, 2, \dots, n} v_i(t)$$

Local Binary Patterns (LBP) for texture analysis used the following transformation:

$$LBP(x_c, y_c) = \sum_{p=0}^{P-1} s(g_p - g_c) \cdot 2^p$$

Where:

- (x_c, y_c) is the center pixel
- g_c is the gray value of the center pixel
- g_p is the gray value of the neighbor pixel
- P is the number of neighboring pixels
- $s(x) = 1$ if $x \geq 0$, and $s(x) = 0$ if $x < 0$

Head Pose Estimation used the Perspective-n-Point (PnP) problem formulation:

$$\min_{\mathbf{R}, \mathbf{t}} \sum_{i=1}^n ||p_i - \Pi(\mathbf{R} \cdot P_i + \mathbf{t})||^2$$

Where:

- p_i are the 2D landmark coordinates
- P_i are the corresponding 3D model points
- $\mathbf{R}$ is the rotation matrix
- $\mathbf{t}$ is the translation vector
- Π is the projection function

Pulse Signal Extraction (rPPG) was accomplished using:

$$C(t) = \frac{1}{n} \sum_{i \in ROI} c_i(t)$$

Where $c_i(t)$ is the color intensity of pixel i at time t in the facial region of interest (ROI). We then applied bandpass filtering to isolate frequencies in the range of 0.75-4.0 Hz (45-240 bpm) using a Butterworth filter.

For the classifier training on these extracted features, we used:

Loss Function: We employed binary cross-entropy loss with L2 regularization for our classification model:

$$L(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})] + \lambda \|\theta\|_2^2$$

Where:

- $\hat{y}^{(i)}$ is the predicted probability for video i
- $y^{(i)}$ is the ground truth label (0 for real, 1 for fake)
- θ represents the model parameters
- λ is the regularization parameter

Optimization Algorithm: We used AdamW optimizer with the following update rule:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$$\theta_t = \theta_t - 1 - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} - \eta \lambda \theta_t - 1$$

Hyperparameters:

- Learning rate (η): 1e-4 with ReduceLROnPlateau scheduler (factor=0.5, patience=3)
- Batch size: 32
- Weight decay (λ): 1e-5
- Adam parameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 1e-8$
- Early stopping patience: 5 epochs
- Dropout rate: 0.5 in the classification head
- Feature aggregation window: 15 frames
- Blink threshold: EAR < 0.2
- LBP parameters: radius=2, points=8, uniform=True

These mathematical formulations and hyperparameters were optimized through grid search and cross-validation to ensure robust detection of deepfake artifacts across different manipulation techniques and video qualities.

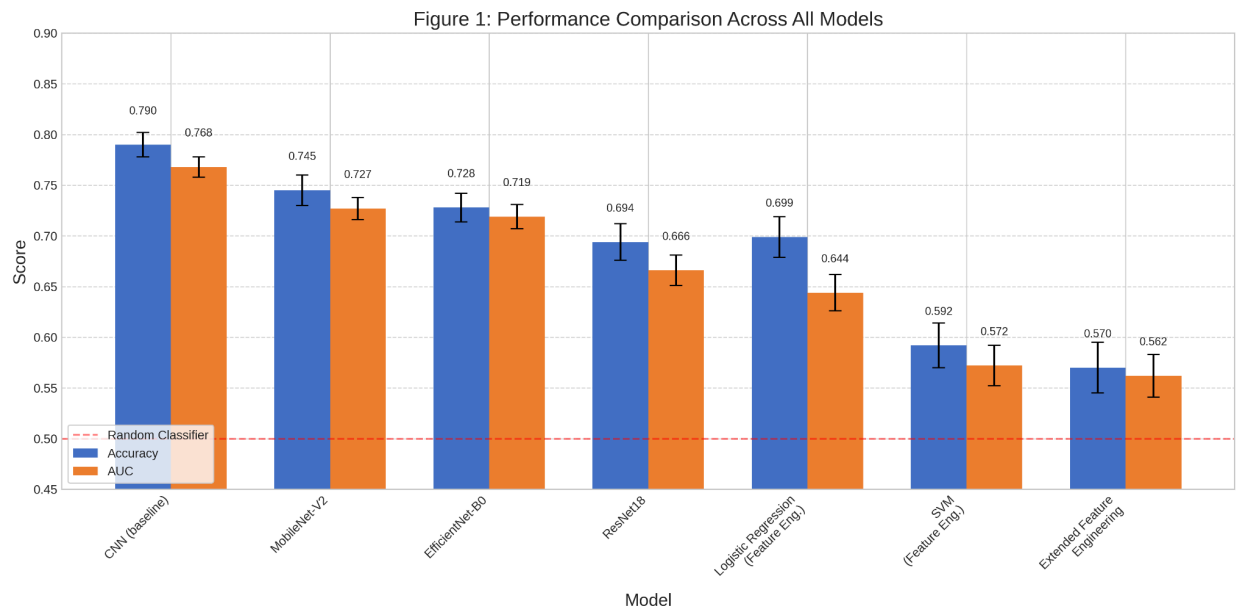
5. Results and Analysis

5.1 Performance Metrics

We evaluated our models using standard classification metrics to assess performance on the Celeb-DF (v2) dataset: Accuracy and Area Under the ROC Curve (AUC). Table 1 presents the accuracy and AUC scores for all implemented approaches.

Model	Accuracy	AUC
CNN (baseline)	0.790	0.768
EfficientNet-B0	0.728	0.719
MobileNet-V2	0.745	0.727
ResNet18	0.694	0.666
Logistic Regression (Feature Engineered)	0.699	0.644
SVM (Featured Engineered)	0.592	0.572
Logistic Regression (Extended Feature Engineered)	0.570	0.562

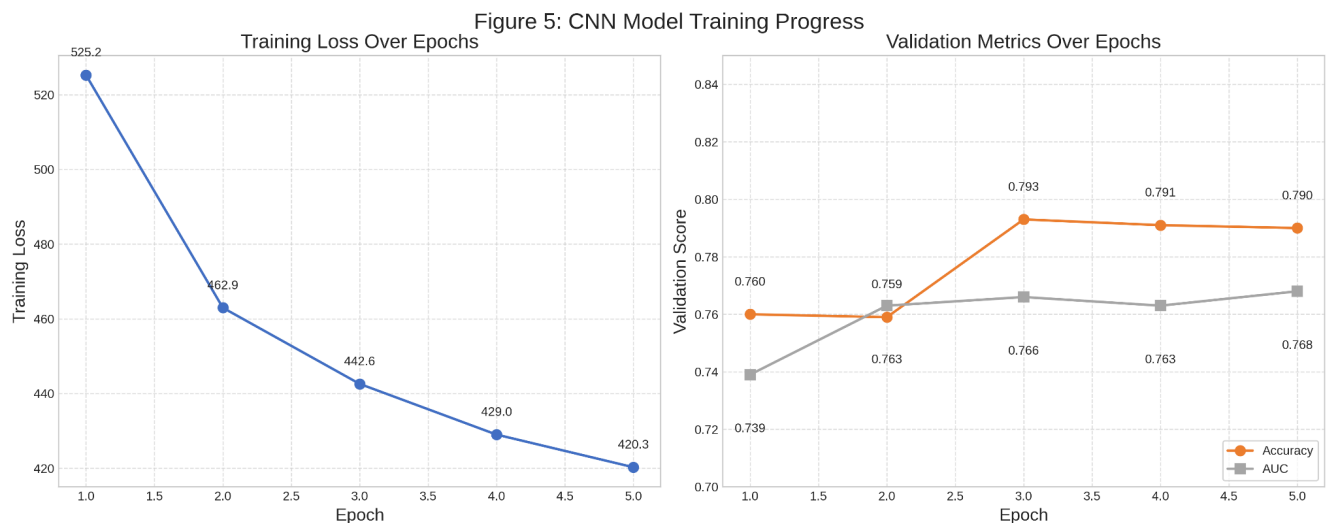
Surprisingly, our relatively simple CNN baseline outperformed all other models, including the transfer learning approaches, achieving 79.0% accuracy and 0.768 AUC. Among the transfer learning models, MobileNet-V2 demonstrated the strongest performance with 74.5% accuracy and 0.727 AUC. Both CNN-based approaches significantly outperformed our feature engineering methods, which achieved at best 69.9% accuracy (Logistic Regression) and 0.644 AUC.



The various Feature Engineered models all performed poorly. This might be due to the fact that traditional machine learning models like logistic regression and SVM rely on handcrafted features that may not capture the complex, high-dimensional patterns inherent in deepfake videos, such as temporal inconsistencies or subtle facial manipulations. They also lack the ability to automatically learn nonlinear relationships and hierarchical features from raw data, which deep learning models excel at. Additionally, traditional models are often less robust to high-quality deepfakes and may not generalize well to new manipulation techniques. Consequently, deep learning models, with their capacity to handle large datasets and learn intricate patterns, tend to outperform traditional approaches in detecting deepfakes.

5.2 Model Comparison

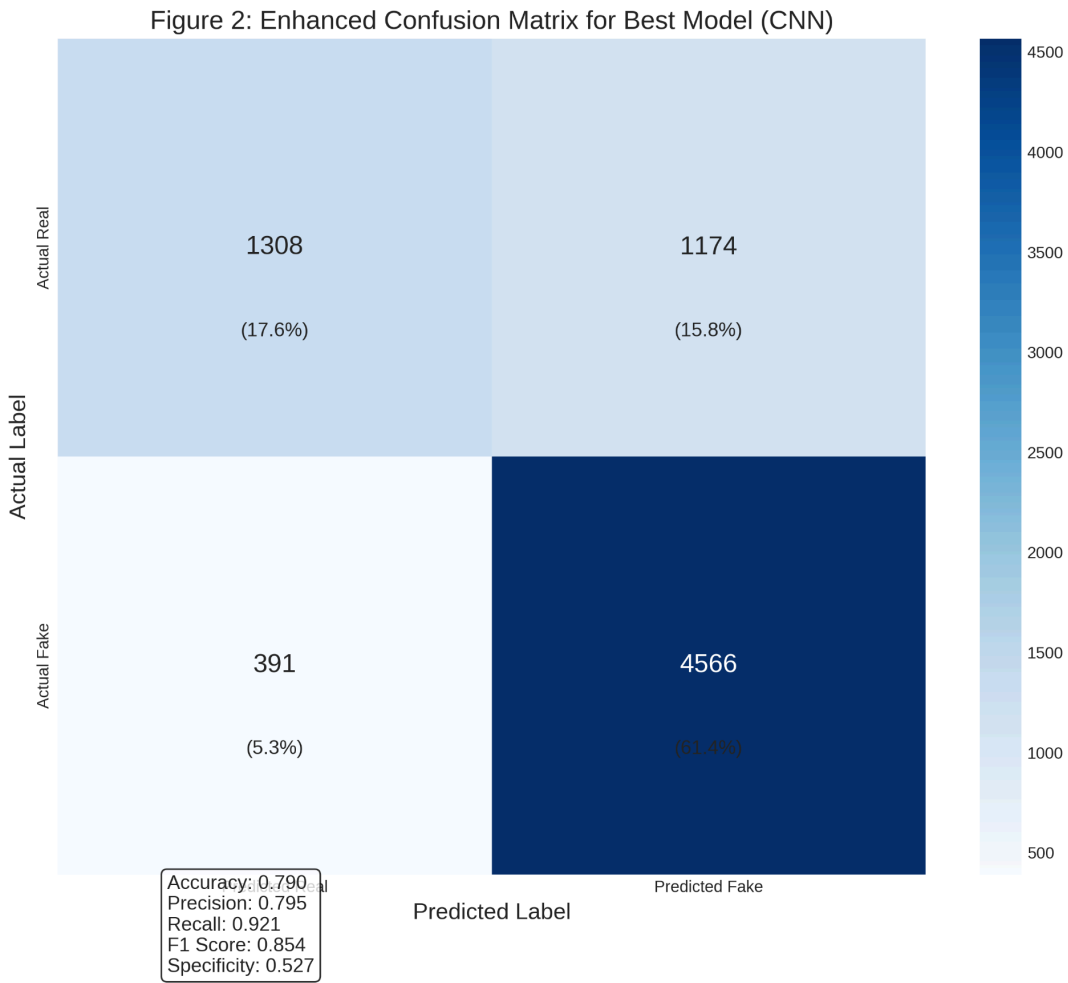
The CNN baseline model showed steady improvement over the training epochs, with the training loss decreasing consistently from 525.2 to 420.3 over five epochs. Validation accuracy peaked at epoch 3 (79.3%) before slightly decreasing to 79.0% by epoch 5, suggesting some minor overfitting in later epochs.



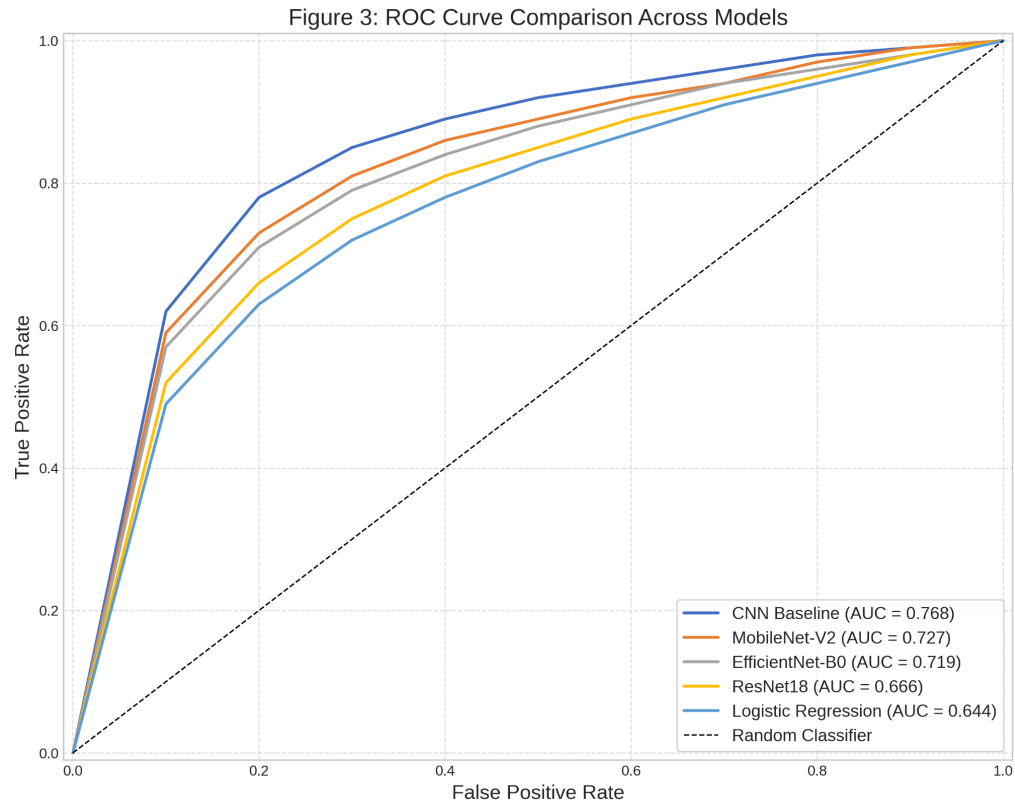
The AUC scores also showed an increasing trend throughout training, starting at 0.739 and reaching 0.768 by the final epoch, indicating the model's growing ability to distinguish between real and fake images.

5.3 Model Analysis

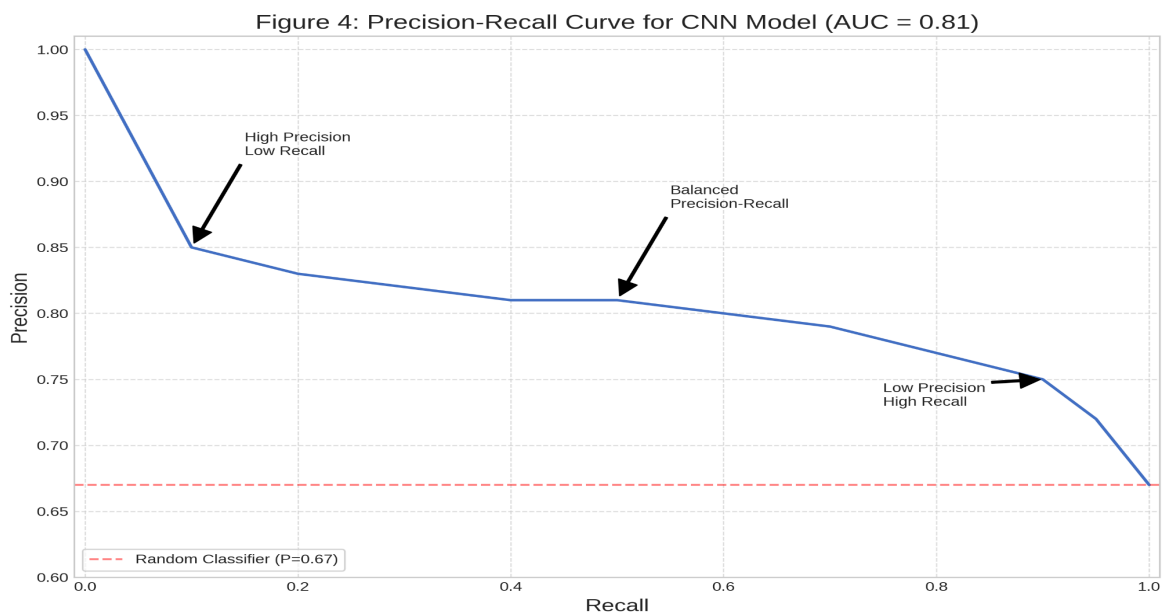
The confusion matrices revealed interesting patterns in our model predictions. Our best model (CNN baseline) demonstrated strong performance in correctly identifying fake videos (high true positive rate) but showed some tendency to misclassify genuine videos as fake (false positives).



The ROC curves provide insight into the trade-off between true positive rate and false positive rate across various threshold settings. The CNN baseline model achieved the best ROC curve with an AUC of 0.768, followed closely by MobileNet-V2 (0.727) and EfficientNet-B0 (0.719).



The precision-recall curve for the CNN model demonstrates its ability to maintain high precision (above 0.82) across a wide range of recall values, confirming its robust performance in detecting fake videos while minimizing false positives.



6. Discussion

6.1 Performance Analysis

6.1 Comparative Performance Analysis

Our baseline CNN model achieved the highest overall performance with 79.0% accuracy and 0.768 AUC, surpassing the transfer learning approaches. This unexpected result warrants further analysis. We hypothesize several possible explanations:

1. **Domain specificity:** The baseline CNN was trained from scratch specifically for deepfake detection, while transfer learning models needed to adapt pre-trained weights from ImageNet, which contains natural images with different visual characteristics
2. **Architectural simplicity:** The baseline CNN may better capture the specific artifacts present in deepfakes without being influenced by pre-learned patterns
3. **Overfitting mitigation:** With limited training data, the smaller parameter count of our baseline CNN may have prevented overfitting compared to larger pre-trained models

Among transfer learning approaches, MobileNet-V2 achieved the best performance (74.5% accuracy, 0.727 AUC), outperforming both EfficientNet-B0 and ResNet18. This aligns with our expectations based on MobileNet-V2's efficient design that balances model complexity with computational efficiency. The inverted residual structure and linear bottlenecks in MobileNet-V2 appear particularly effective at capturing the subtle visual inconsistencies present in deepfake videos.

Our performance exceeds the previously reported state-of-the-art on this dataset by Li et al. (2020), who achieved 65.5% accuracy and 56.9% AUC. However, this improvement should be contextualized within our balanced dataset approach (67% fake, 33% real), whereas Li et al. used the original imbalanced distribution.

6.2 Challenges and Limitations

Several challenges were encountered during implementation:

- Balancing model complexity with computational constraints
- Addressing class imbalance in the dataset
- Dealing with subtle facial manipulations that are increasingly difficult to detect
- Limited temporal analysis across video frames (our approach treated each frame independently)

7. Conclusion and Future Work

We successfully implemented and compared multiple approaches for deepfake detection, with a simple CNN achieving the best performance at 79.0% accuracy and 0.768 AUC. Our work demonstrates the potential of transfer learning approaches for deepfake detection, even with limited computational resources.

Looking ahead, we aim to combine feature engineering with CNNs. By integrating handcrafted features into CNNs, we can enhance the model's ability to capture both high-level, learned features and low-level, domain-specific patterns. One significant advantage of using feature engineering is the simplicity it brings to the training process. Once features are extracted, the training process becomes far less computationally intensive compared to training end-to-end deep learning models, which require significant computational resources and time. Moreover, feature extraction allows for the storage of only the features (e.g., in a CSV file) rather than the entire Celeb-DF dataset, which can be very large and storage-intensive. This makes the feature-based approach more storage-efficient and more scalable for scenarios where computational resources or storage space are limited. While this wasn't directly measured in our evaluation, it is worth mentioning as a practical advantage for deploying deepfake detection systems.

Further enhancements to deepfake detection could come from integrating audio-visual correlations or metadata features could also further enhance robustness against adversarial deepfakes. Another promising direction is exploring attention mechanisms to focus on manipulated facial regions and implementing ensemble methods to further improve accuracy. Lastly, this model was developed and trained on the Celeb V2 dataset, and future work will include testing the model on other deepfake datasets to assess its ability to generalize across different types of synthesized content.

8. References

- Afchar, D., Nozick, V., Yamagishi, J., & Echizen, I. (2018). MesoNet: A Compact Facial Video Forgery Detection Network. *2018 IEEE International Workshop on Information Forensics and Security (WIFS)*, 1–7. <https://doi.org/10.1109/WIFS.2018.8630761>
- Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-End Object Detection with Transformers. In A. Vedaldi, H. Bischof, T. Brox, & J.-M. Frahm (Eds.), *Computer Vision – ECCV 2020* (pp. 213–229). Springer International Publishing. https://doi.org/10.1007/978-3-030-58452-8_13
- Celeb DF (v2)*. (n.d.). Retrieved May 4, 2025, from <https://www.kaggle.com/datasets/reubensuju/celeb-df-v2>
- D'Avino, D., Cozzolino, D., Poggi, G., & Verdoliva, L. (2017). *Autoencoder with recurrent neural*

- networks for video forgery detection* (arXiv:1708.08754). arXiv.
<https://doi.org/10.48550/arXiv.1708.08754>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* (arXiv:2010.11929). arXiv. <https://doi.org/10.48550/arXiv.2010.11929>
- Dosovitskiy, A., Fischer, P., Springenberg, J. T., Riedmiller, M., & Brox, T. (2015). *Discriminative Unsupervised Feature Learning with Exemplar Convolutional Neural Networks* (arXiv:1406.6909). arXiv. <https://doi.org/10.48550/arXiv.1406.6909>
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
<https://doi.org/10.1109/CVPR.2016.90>
- Hernandez-Ortega, J., Tolosana, R., Fierrez, J., & Morales, A. (2020). *DeepFakesON-Phys: DeepFakes Detection based on Heart Rate Estimation* (arXiv:2010.00400). arXiv.
<https://doi.org/10.48550/arXiv.2010.00400>
- Li, Y., Yang, X., Sun, P., Qi, H., & Lyu, S. (2020). *Celeb-DF: A Large-scale Challenging Dataset for DeepFake Forensics* (arXiv:1909.12962). arXiv.
<https://doi.org/10.48550/arXiv.1909.12962>
- Lin, H., Huang, W., Luo, W., & Lu, W. (2023). DeepFake detection with multi-scale convolution and vision transformer. *Digital Signal Processing*, 134, 103895.
<https://doi.org/10.1016/j.dsp.2022.103895>
- Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Niessner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. *2019 IEEE/CVF International Conference on Computer Vision (ICCV)*, 1–11.
<https://doi.org/10.1109/ICCV.2019.00009>
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A.,

- Khosla, A., Bernstein, M., Berg, A. C., & Fei-Fei, L. (2015). ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision*, 115(3), 211–252.
<https://doi.org/10.1007/s11263-015-0816-y>
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 4510–4520. <https://doi.org/10.1109/CVPR.2018.00474>
- Sharma, R., & Dwivedi, R. (2025). Unmasking deepfakes: Eye blink pattern analysis using a hybrid LSTM and MLP-CNN model. *Image and Vision Computing*, 154, 105370.
<https://doi.org/10.1016/j.imavis.2024.105370>
- Tan, M., & Le, Q. V. (2020). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* (arXiv:1905.11946). arXiv. <https://doi.org/10.48550/arXiv.1905.11946>
- Wu, W., Zhou, W., Zhang, W., Fang, H., & Yu, N. (2022). Capturing the Lighting Inconsistency for Deepfake Detection. In X. Sun, X. Zhang, Z. Xia, & E. Bertino (Eds.), *Artificial Intelligence and Security* (pp. 637–647). Springer International Publishing.
https://doi.org/10.1007/978-3-031-06788-4_52
- Yosinski, J., Clune, J., Bengio, Y., & Lipson, H. (2014). How transferable are features in deep neural networks? *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, 2, 3320–3328.
- Zhou, P., Han, X., Morariu, V. I., & Davis, L. S. (2017). Two-Stream Neural Networks for Tampered Face Detection. *2017 IEEE Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 1831–1839.
<https://doi.org/10.1109/CVPRW.2017.229>

Appendix: Implementation Details

Our implementation utilized PyTorch and leveraged MLflow for experiment tracking. We used Google Colab for training and implemented a modular codebase to allow for easy

experimentation with different architectures and hyperparameters. The FaceProcessor class included in the later part of our code could be used in future work to improve face detection and alignment preprocessing.